

Aprendizado por Reforço em Ambiente Grid-World para Navegação e Evasão de Obstáculos

Implementação de Arquitetura A2C-LSTM em Entornos
GridWorld Dinâmicos

Brithany Michelle Oliva Chuquimia
Luis Enrique Cardozo Ramirez



Introdução ao Problema



O Problema

Robôs móveis e veículos autônomos operam em ambientes dinâmicos onde devem planejar trajetórias seguras. O desafio é alcançar um objetivo evitando colisões sem mapas pré-definidos



A Solução: RL

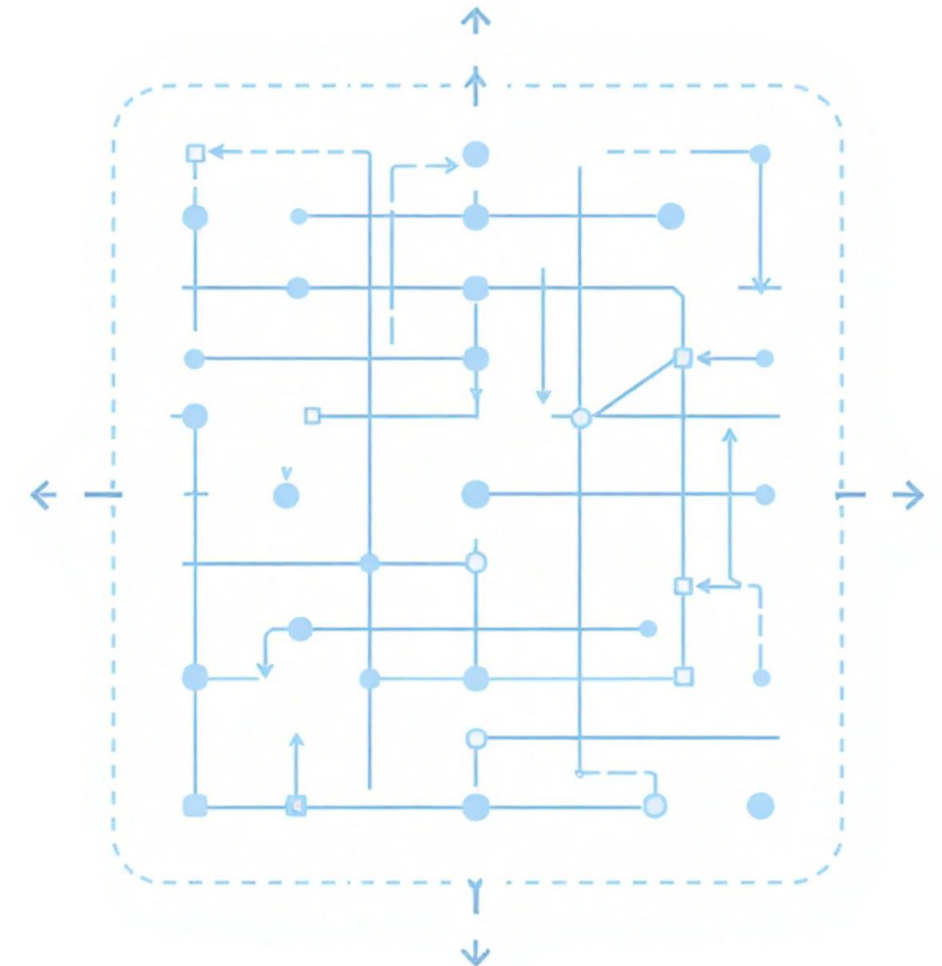
O **Aprendizado por Reforço** permite que um agente aprenda por tentativa e erro. Utilizamos um ambiente *Grid-World* como laboratório para estudar convergência e estratégias de navegação antes de aplicações reais.



Objetivo

Desenvolver agente RL que aprenda políticas eficazes para alcançar metas evitando colisões.

GRID NAVIGATION



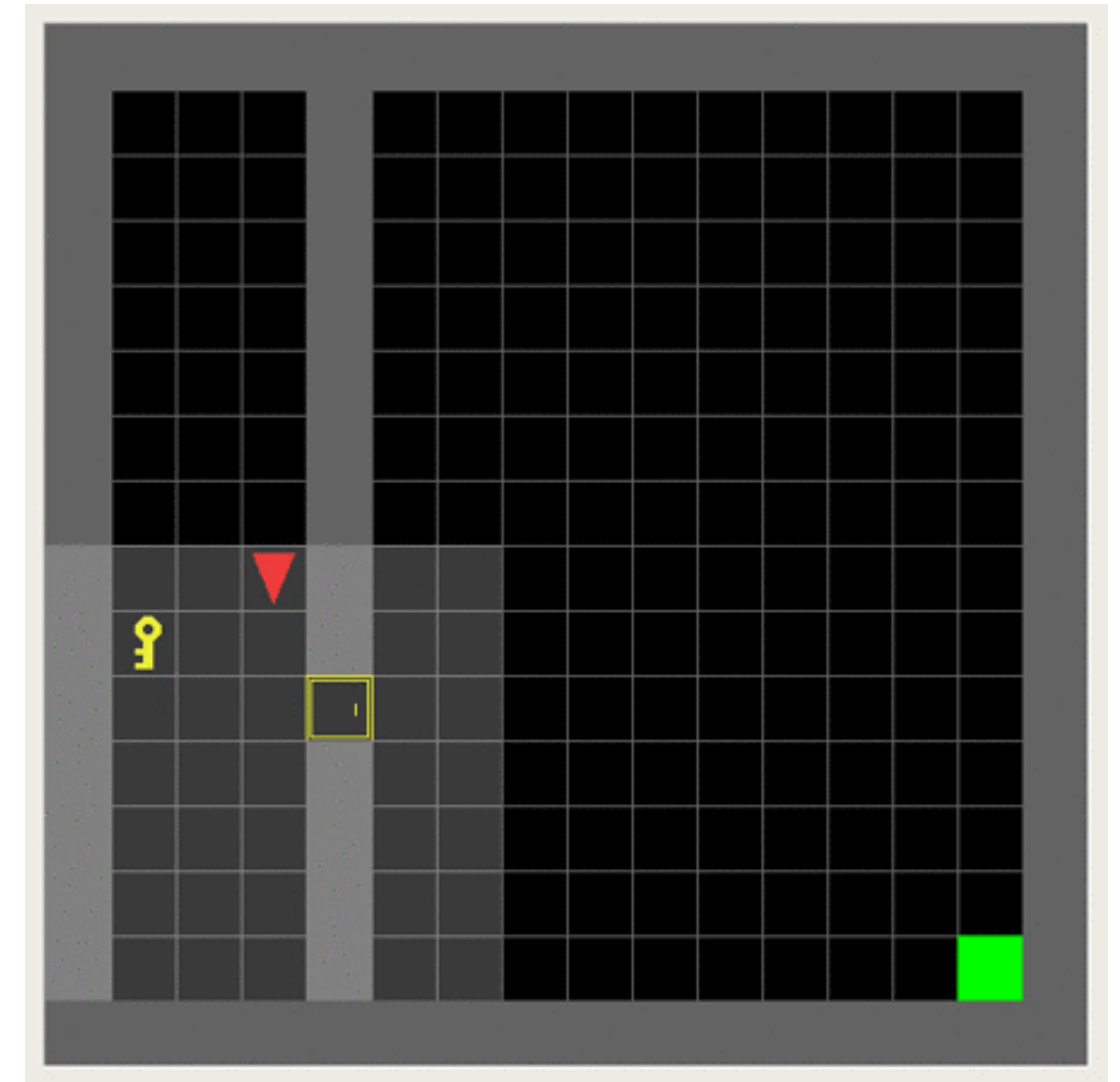
AGRID

Trabalhos Relacionados

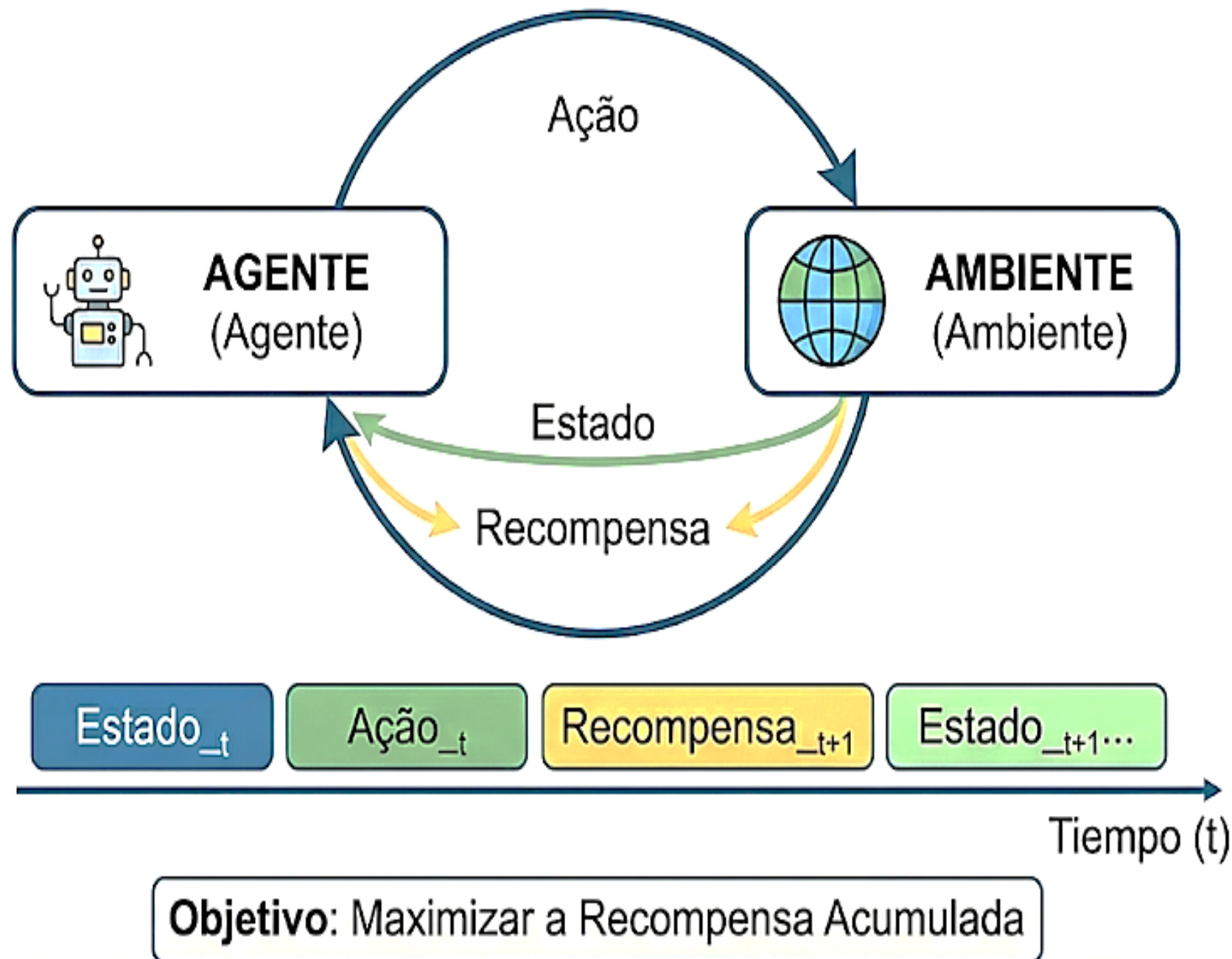
O uso de RL em navegação é detalhado por Sutton e Barto2018. que formalizam o problema como um Processo de Decisão de Markov e estabelecem as bases para os métodos de Avaliação de Políticas e Controle.

A opção pelo algoritmo Actor-Critic é motivada pela sua capacidade de aprender simultaneamente a função de valor e a política de ação, conforme revisado em detalhes por Grondman et al.2012.

O trabalho de Kochenderfer et al. 2023 sobre algoritmos de tomada de decisão reforça a relevância do problema proposto ao tratar a navegação e a otimização de caminhos como um problema central em sistemas autônomos.



Fundamentos do Aprendizado por Reforço (RL)



O Aprendizado por Reforço é um paradigma no qual um agente aprende por interação direta com um ambiente dinâmico. A cada passo, o agente observa um estado, escolhe uma ação e recebe uma recompensa que avalia a qualidade dessa decisão.

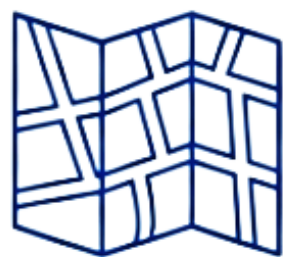
Ciclo de Aprendizado

Observação → Ação →
Recompensa → Atualização

Vantagens

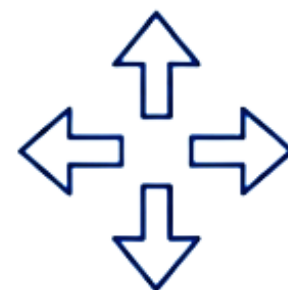
Descoberta autônoma
de rotas ótimas em
ambientes dinâmicos.

Fundamentos do Aprendizado por Reforço (RL)



Estado (S)

Uma representação do ambiente em um determinado momento. É a informação que o agente usa para decidir.



Ações (A)

O conjunto de decisões que o agente pode tomar. Em nosso caso: mover para cima, baixo, esquerda ou direita.



Recompensa (R)

O sinal de feedback do ambiente. Informa ao agente se a última ação foi boa ou ruim, guiando o aprendizado.



Política (π)

O "cérebro" do agente. É a estratégia que mapeia estados para ações, definindo como o agente se comporta.

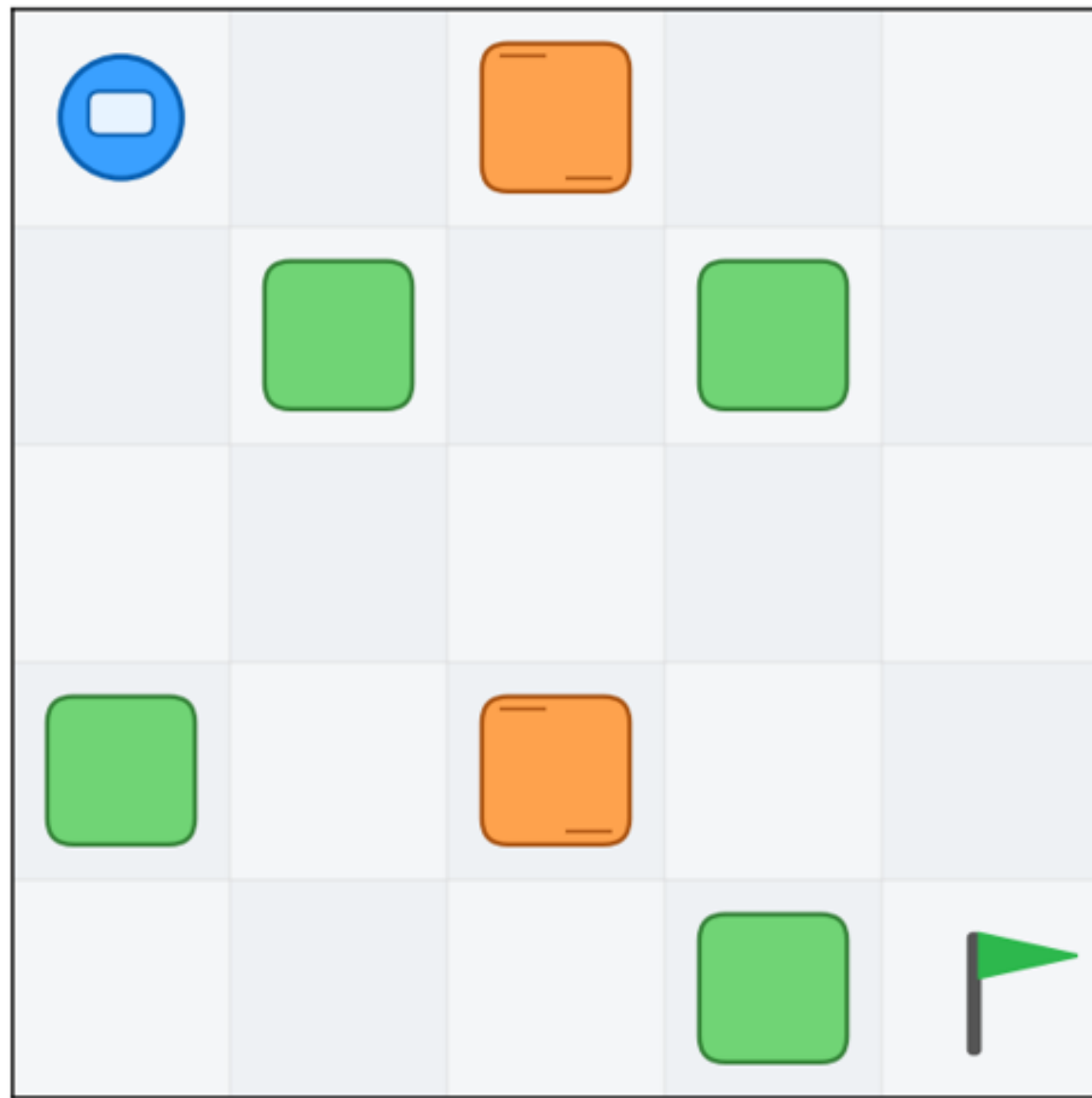


Valor (V)

A previsão da recompensa futura acumulada que o agente espera receber a partir de um estado. Ajuda a avaliar a "qualidade" de estar em um determinado estado.

O objetivo do RL é encontrar uma **Política** que maximize o **Valor** esperado, ou seja, a recompensa acumulada ao longo do tempo.

Metodologia: Entorno Experimental: GridWorld



Agente Meta Obs. estático Obs. dinâmico

Ambiente GridWorld 5×5:

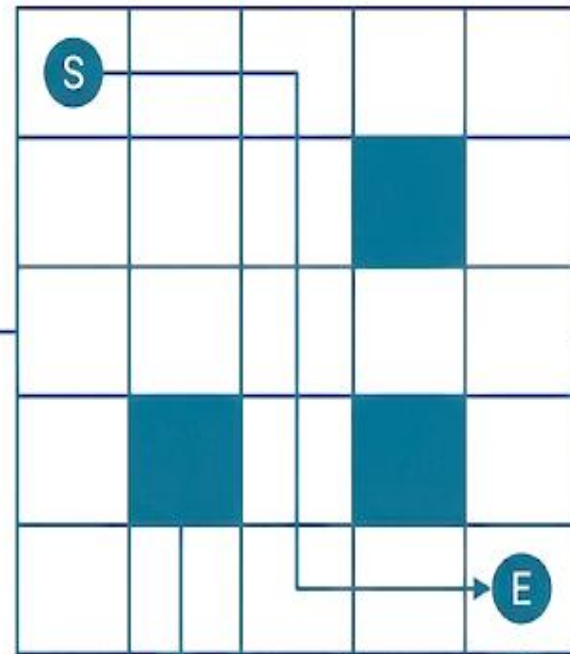
O cenário é estruturado como uma matriz bidimensional projetada para tarefas de navegação e evasão de obstáculos. Cada célula do grid representa:

- **Espaço livre (0):** área navegável;
- **Obstáculo (-1):** colisão resulta em penalização;
- **Meta (2):** posição objetivo que encerra o episódio com recompensa positiva.

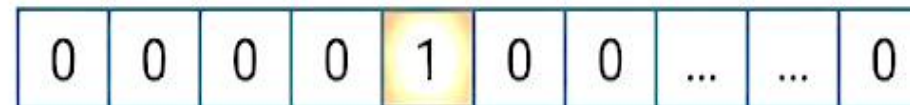
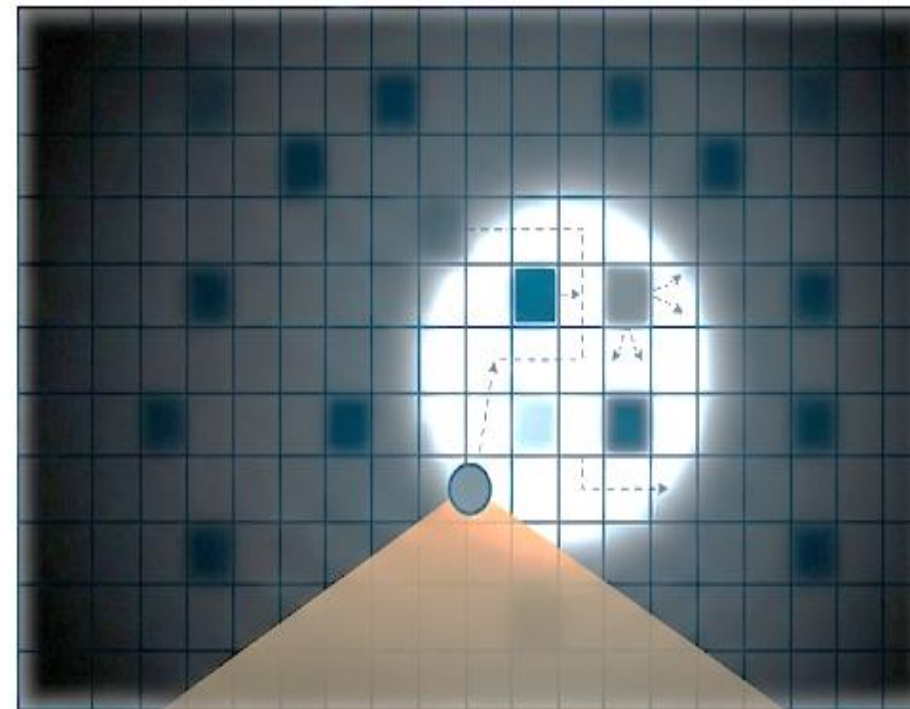
O ambiente inclui **obstáculos estáticos** e **dinâmicos**, estes últimos movendo-se de forma aleatória ou seguindo padrões. Toda a implementação foi realizada em Python no Google Colab, com arquitetura parametrizável para ajuste de dimensões, recompensas e configuração do mapa.”

Metodologia: Entorno Experimental: GridWorld

Modo MDP (Markov Decision Process) –
Observabilidad Completa



Modo POMDP (Partially Observable MDP) –
Visión Restringida + Codificación One-Hot

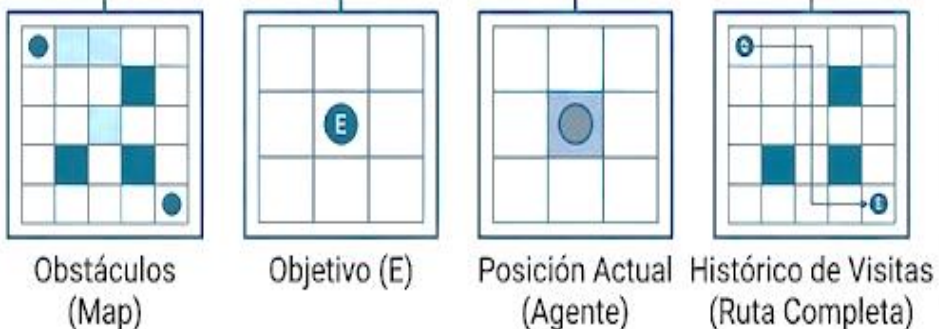


Codificación One-Hot (Posición)

 Visión Local (Limitada)
 Codificación One-Hot (Posición de granyitor vector)

Modos

- MDP (Markov Decision Process):** o agente possui *observabilidade completa* do ambiente. Ele vê todo o mapa, incluindo obstáculos, objetivo, sua posição e o histórico de visitas.
- POMDP (Partially Observable MDP):** o agente enxerga apenas uma *região local* ao seu redor. Para compensar a perda de informação global, utiliza uma **codificação One-Hot** para representar sua posição.
- Diferença central:**
 - MDP → decisões com informação total.
 - POMDP → decisões sob incerteza, exigindo inferência e memória.
- O POMDP aproxima melhor condições reais de navegação em robótica e agentes autônomos.



Arquitetura A2C-LSTM

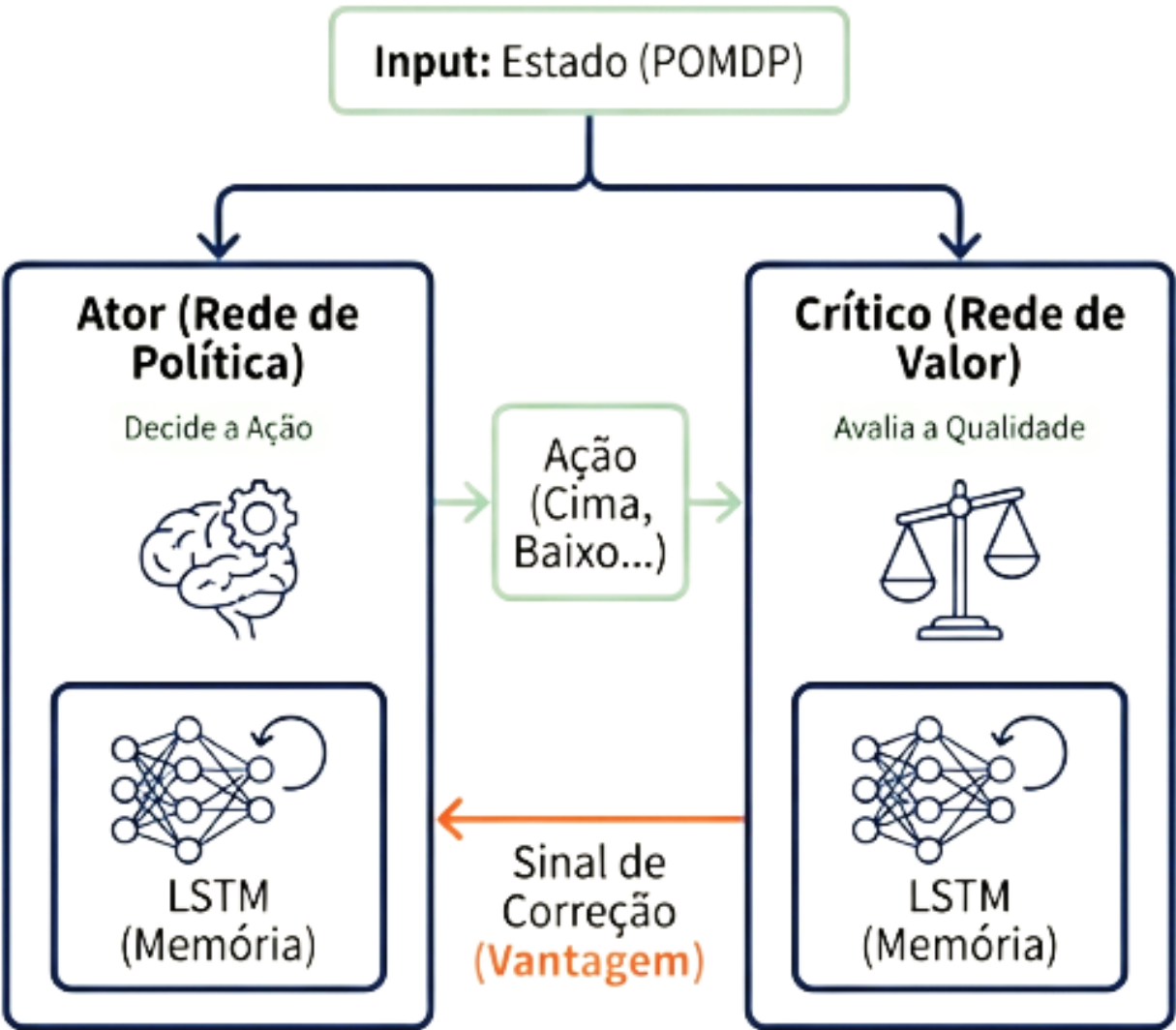
Advantage Actor- Critic (A2C)

Uma arquitetura dual que aprende de forma mais estável e eficiente.

- Ator (Rede de Política):
Decide que ação tomar ($\uparrow \downarrow \leftarrow \rightarrow$) com base no estado atual. É o cérebro de decisão.
- Crítico (Rede de Valor):
Avalia o quão "bom" é o estado atual, ajudando o Ator a refinar suas decisões. É o cérebro de avaliação.

Long Short – Term Memory(LSTM)

Dá ao agente uma "memória" do que viu nos passos anteriores. Se a visão é limitada, a memória é fundamental para construir um mapa do ambiente e tomar decisões informadas.



Componente	Valor
Modo de estado	POMDP
Dimensão do estado	100
Hidden LSTM	128
Ações	4
Ator	LSTM→FC(4)
Crítico	LSTM→FC(1)

Definindo a Motivação: O Design da Função de Recompensa

A forma como recompensamos e penalizamos o agente molda fundamentalmente seu comportamento.
Se teste quatro estratégias distintas para encontrar o equilíbrio ideal.

Equilibrada



Um balanço entre o custo por passo e a recompensa final

Rapidez



Penalizações fortes por passo e recompensa final elevada.
Prioriza trajetos curtos

Exploração



Penalizações suaves, incentivando o agente a explorar sem medo a errar

Segurança



Altas penalizações para colisões, criando um agente mais cauteloso

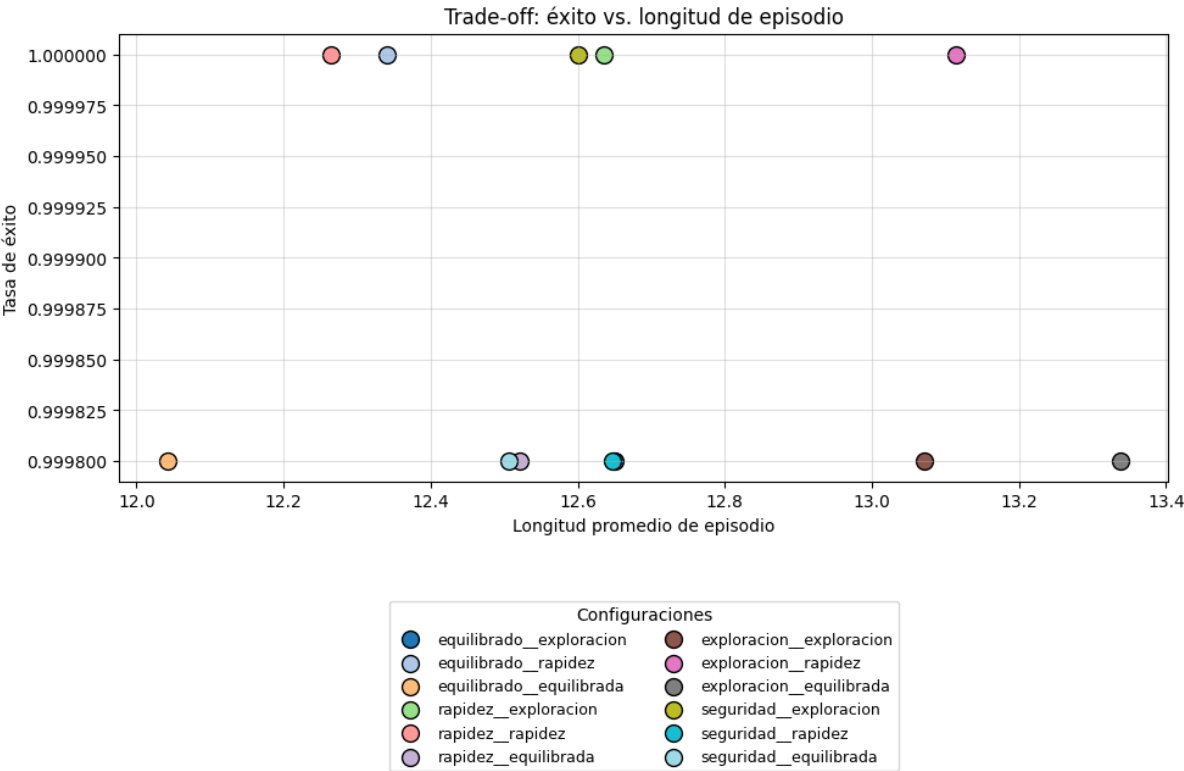
Acelerando o Aprendizado com PBRS

Se uso Potencial-Based Reward Shaping (PBRS) para fornecer um sinal de recompensa mais denso, guiando o agente em direção ao objetivo e acelerando a convergência inicial.

Em Busca da Estratégia Ideal

Para garantir a escolha da configuração mais robusta, cruzamos as 4 funções de recompensa com 3 estratégias de treinamento, que variam principalmente na regularização da entropia (o nível de aleatoriedade da politica).

	Exploratório (alta entropia)	Equilibrado (entropia intermediária)	Voltado à Rapidez (baixa entropia)
Equilibrada	☐ Equilibrada + Exploratório	☐ Equilibrada + Equilibrado	☐ Equilibrada + Voltado à Rapidez
Rapidez	☐ Rapidez + Exploratório	☐ Rapidez + Equilibrado	☐ Rapidez + Voltado à Rapidez
Exploração	☐ Exploração + Exploratório	☐ Exploração + Equilibrado	☐ Exploração + Voltado à Rapidez
Segurança	☐ Segurança + Exploratório	☐ Segurança + Equilibrado	☐ Segurança + Voltado à Rapidez



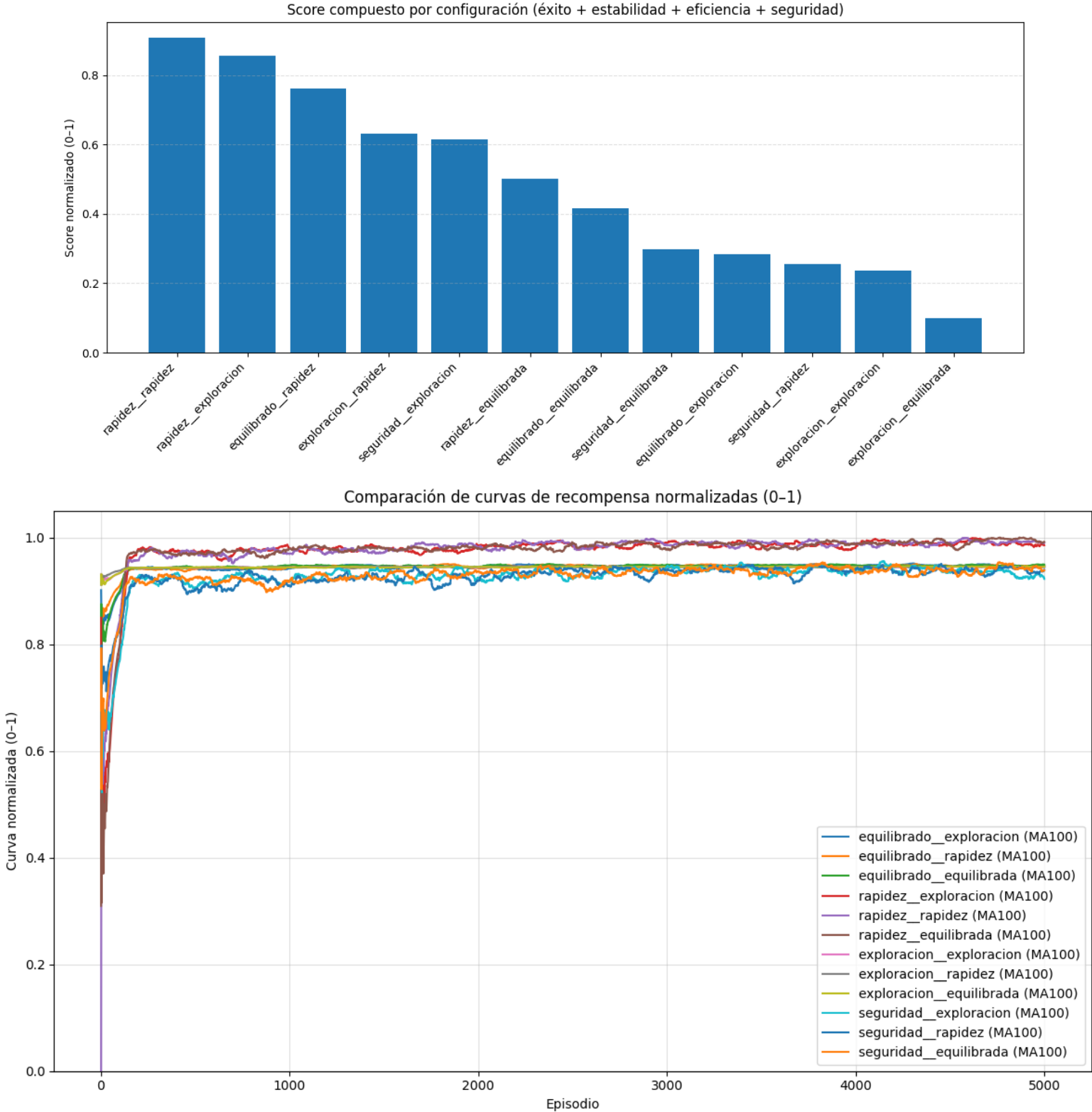
Cada uma das 12 combinações foi avaliada com base na estabilidade da politica, taxa de sucesso, colisões e comprimento médio das trajetórias.

Em Busca da Estratégia Ideal

A combinação de uma função de recompensa do tipo Rapidez com uma estratégia de treinamento igualmente voltada a Rapidez apresentou o melhor desempenho composto em todos os cenários.

	Exploratório (alta entropia)	Equilibrado (entropia intermediária)	Voltado à Rapidez (baixa entropia)
Equilibrada	Equilibrada + Exploratório	Equilibrada + Equilibrado	Equilibrada + Voltado à Rapidez
Rapidez	Rapidez + Exploratório	Rapidez + Equilibrado	✓ Rapidez + Voltado à Rapidez
Exploração	Exploração + Exploratório	Exploração + Equilibrado	Exploração + Voltado à Rapidez
Segurança	Segurança + Exploratório	Segurança + Equilibrado	Segurança + Voltado à Rapidez

Esta configuração demonstrou a maior estabilidade e eficiência, sendo selecionada para todos os experimentos



Detalhes da Arquitetura e Treinamento

Parâmetros

Episódios
5.000

Passos/Episódio
300

γ (desconto)
0.99

GAE (λ)
0.95

Entropia
0.02

Coef. Valor
0.6

Otimizador: Adam

α (ator)	3×10^{-4}
α (crítico)	8×10^{-4}
eps	1×10^{-5}

Recompensas

 Objetivo alcançado **+50.0**

 Colisão com parede **-10.0**

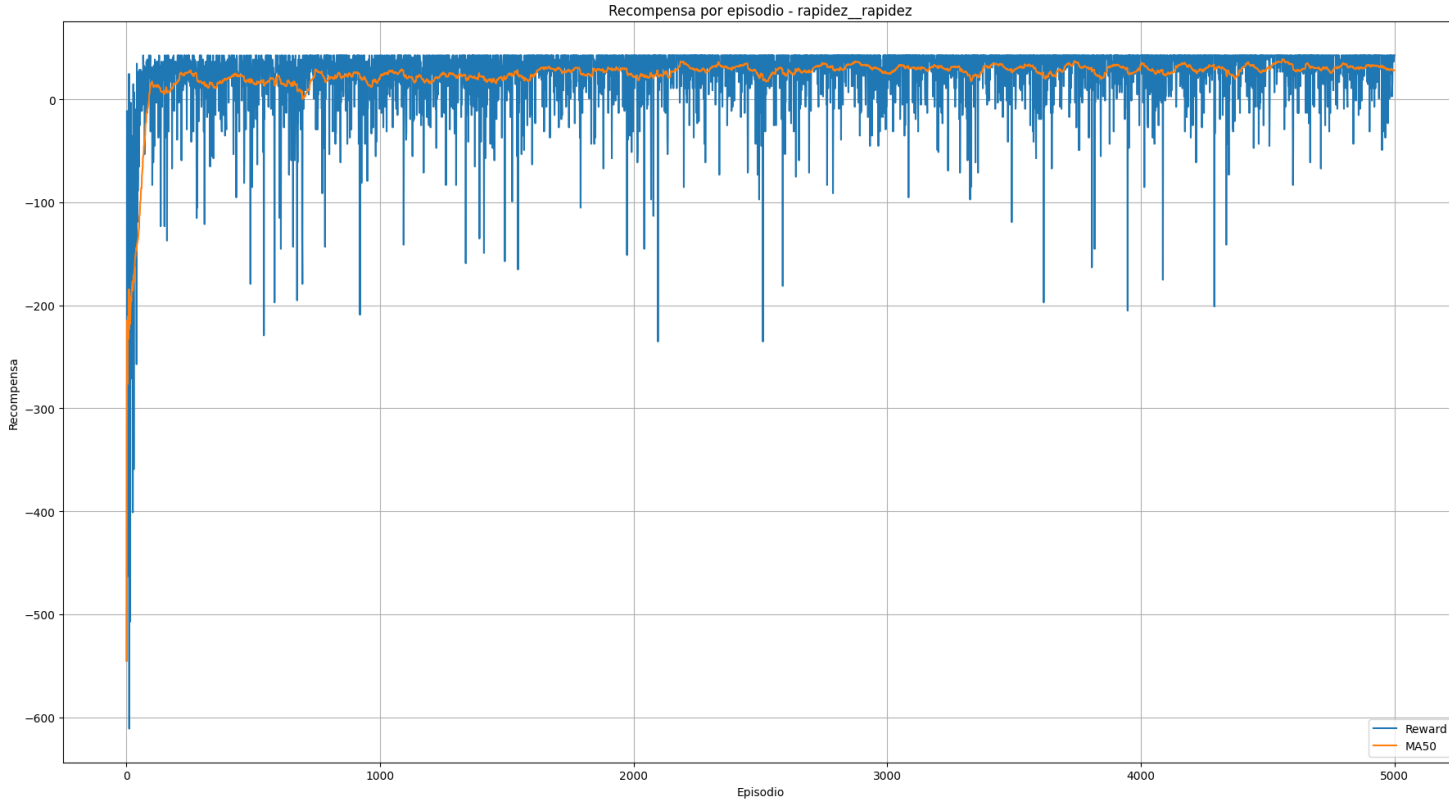
 Sair dos limites **-10.0**

 Passo realizado **-1.0**

 Revisitar célula **-0.05**

 Sem movimento **-1.0**

Resultados de Treinamento



100%

Taxa de Sucesso

Consistente ao longo do treinamento.

8-12

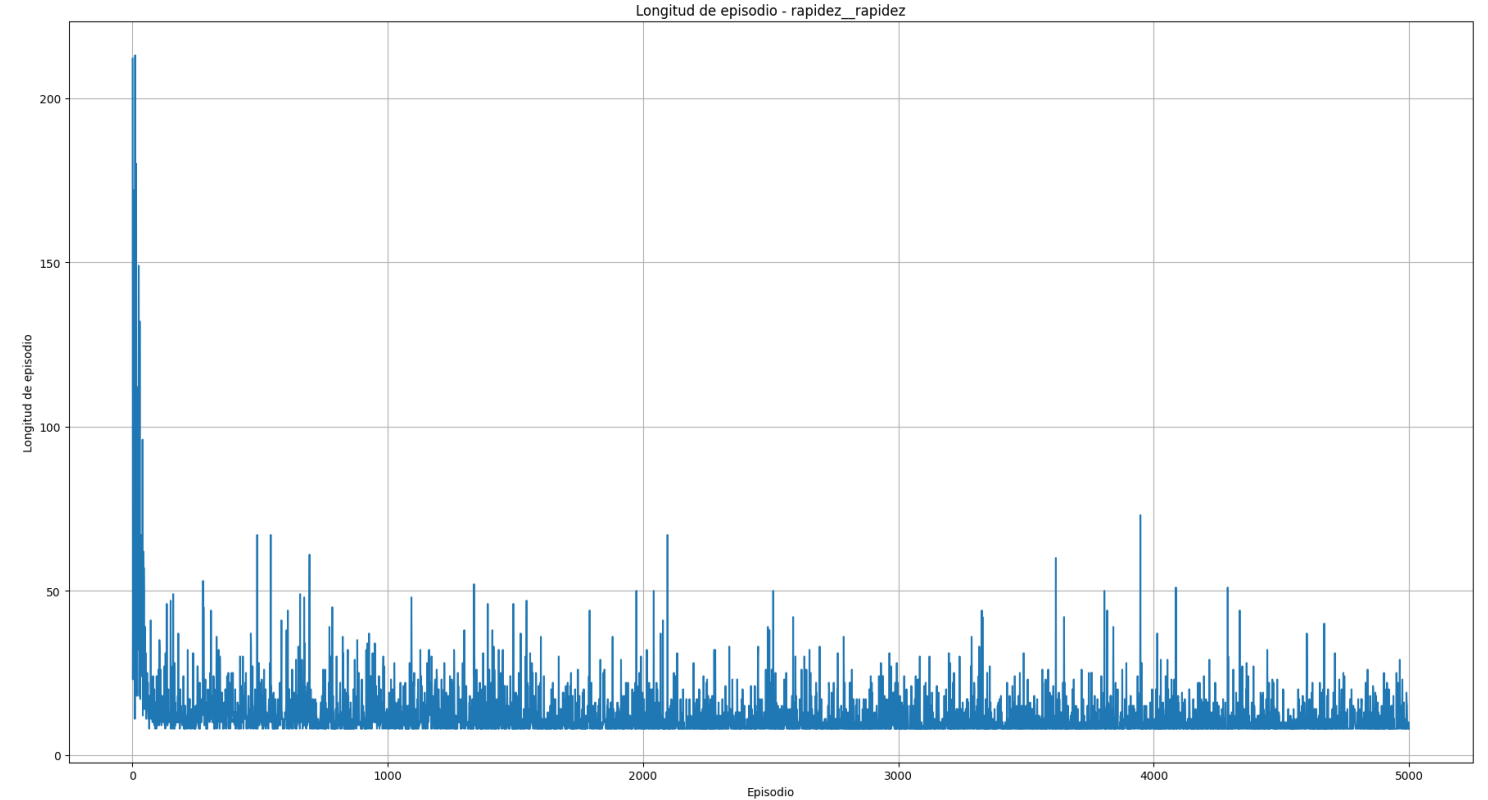
Passos por Episódio

Converge para valores próximos ao ótimo.

+25

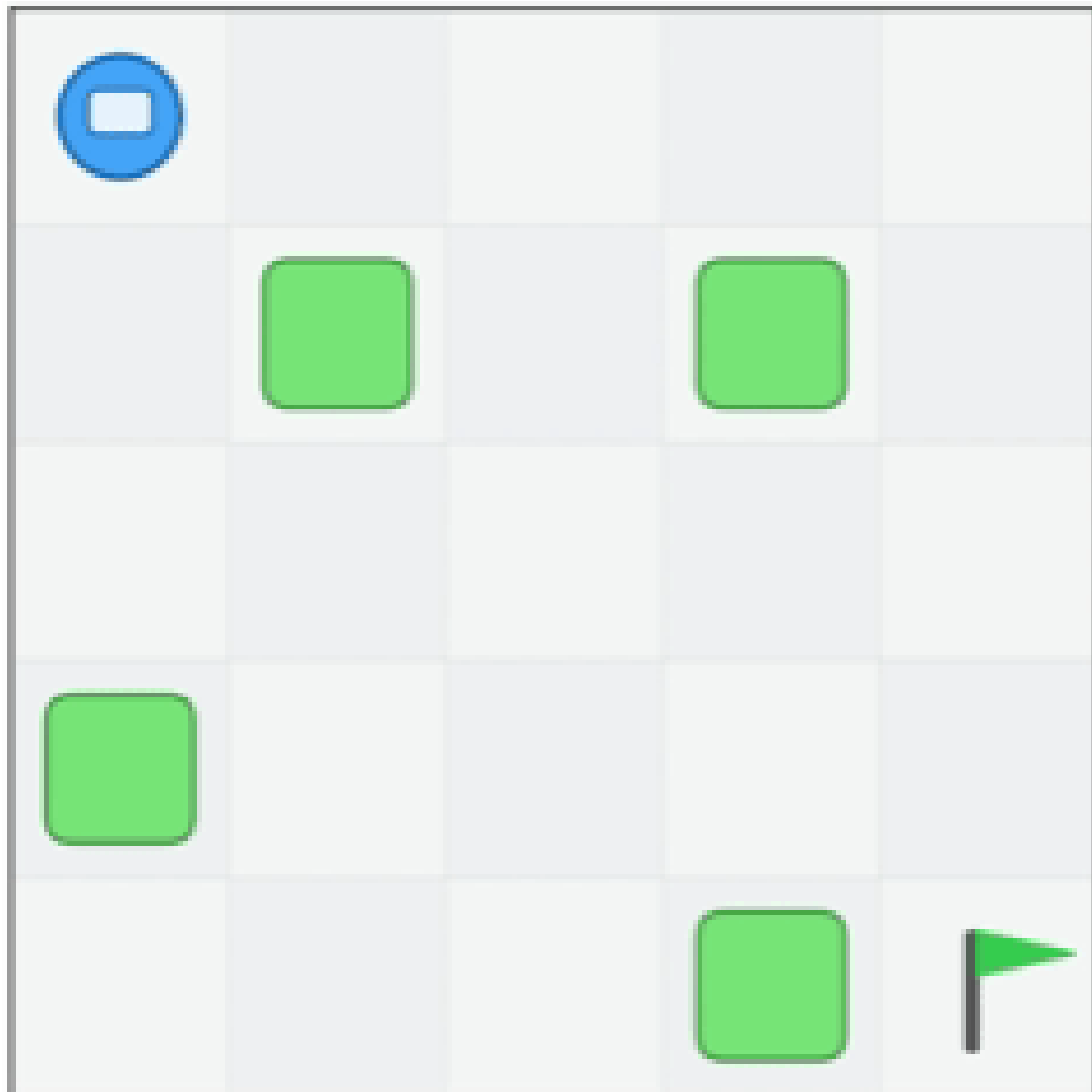
Recompensa MA100

Após 1000 episódios de treinamento.



- **Início Negativo:** As recompensas iniciais são negativas devido a forte penalização por passo da configuração. O agente está explorando e cometendo erros.
- **Ponto de Vista:** A convergência ocorre entre 200 – 300 episódios, a média móvel torna-se positiva.
- **Performance de Platô:** Entre os episódios 1000 e 5000, a performance se estabiliza em um nível alto, com a média móvel frequentemente superando +25 indicando trajetórias curtas e eficientes

Validação: Testes A, B e C



Teste A — Ambiente Estático

ROTA BASE

Avaliação em ambiente sem obstáculos móveis

100%

Taxa de Sucesso

8

Passos

0

Penalizações

Conclusão: Este teste estabelece a rota de referência ("rota base") utilizada nas análises seguintes.

Validação: Testes A, B e C



Teste B — Bloqueio e Reencaminhamento

ADAPTAÇÃO

Capacidade de desvio com obstáculo na rota ótima

9

Trajeto Base

11

Com Bloqueio

+2

Desvio Adicional

4

Passo de Divergência

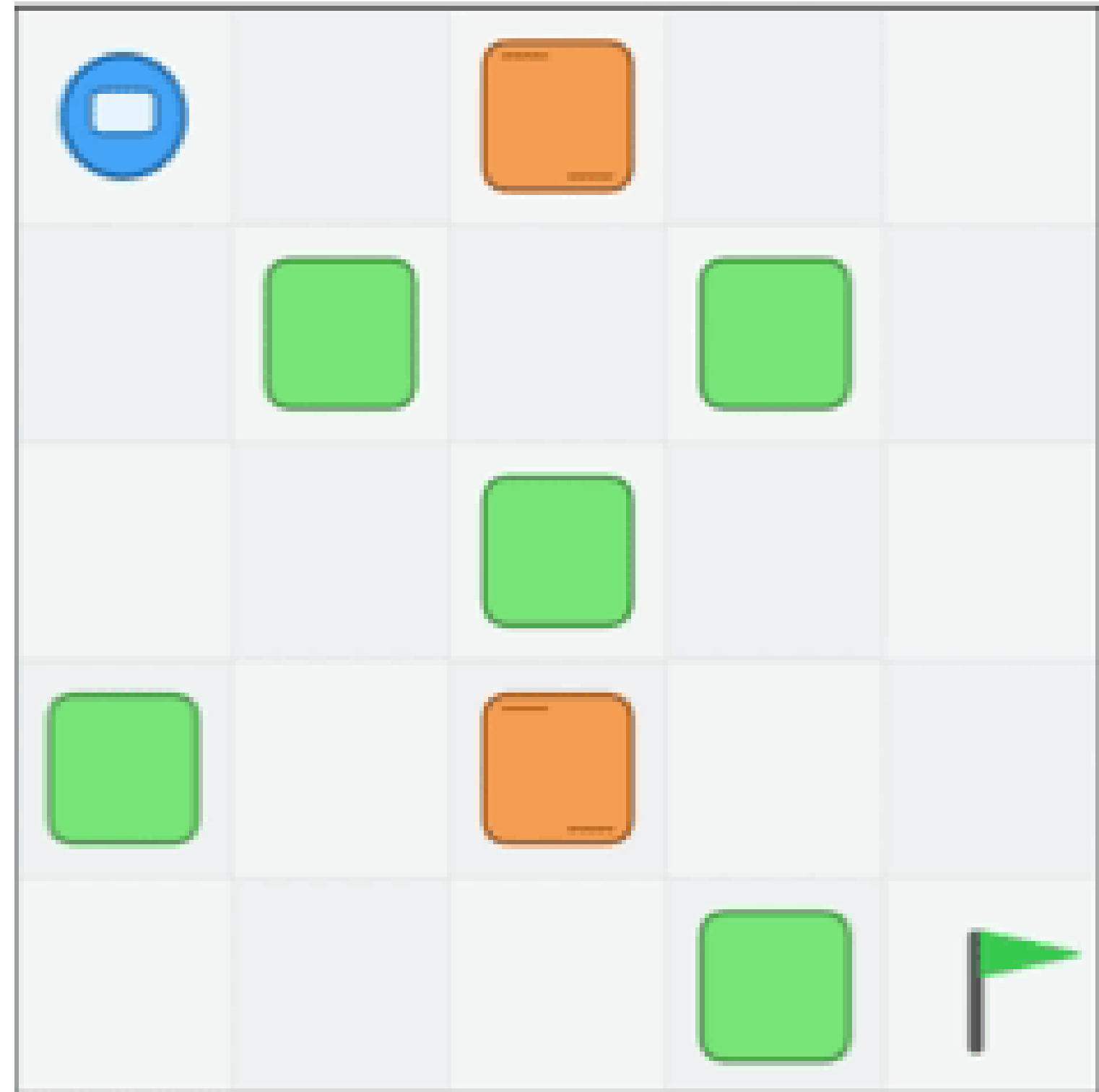
⚠ Colisões Ocasionais

- 2 colisões com parede
- Múltiplas revisitas

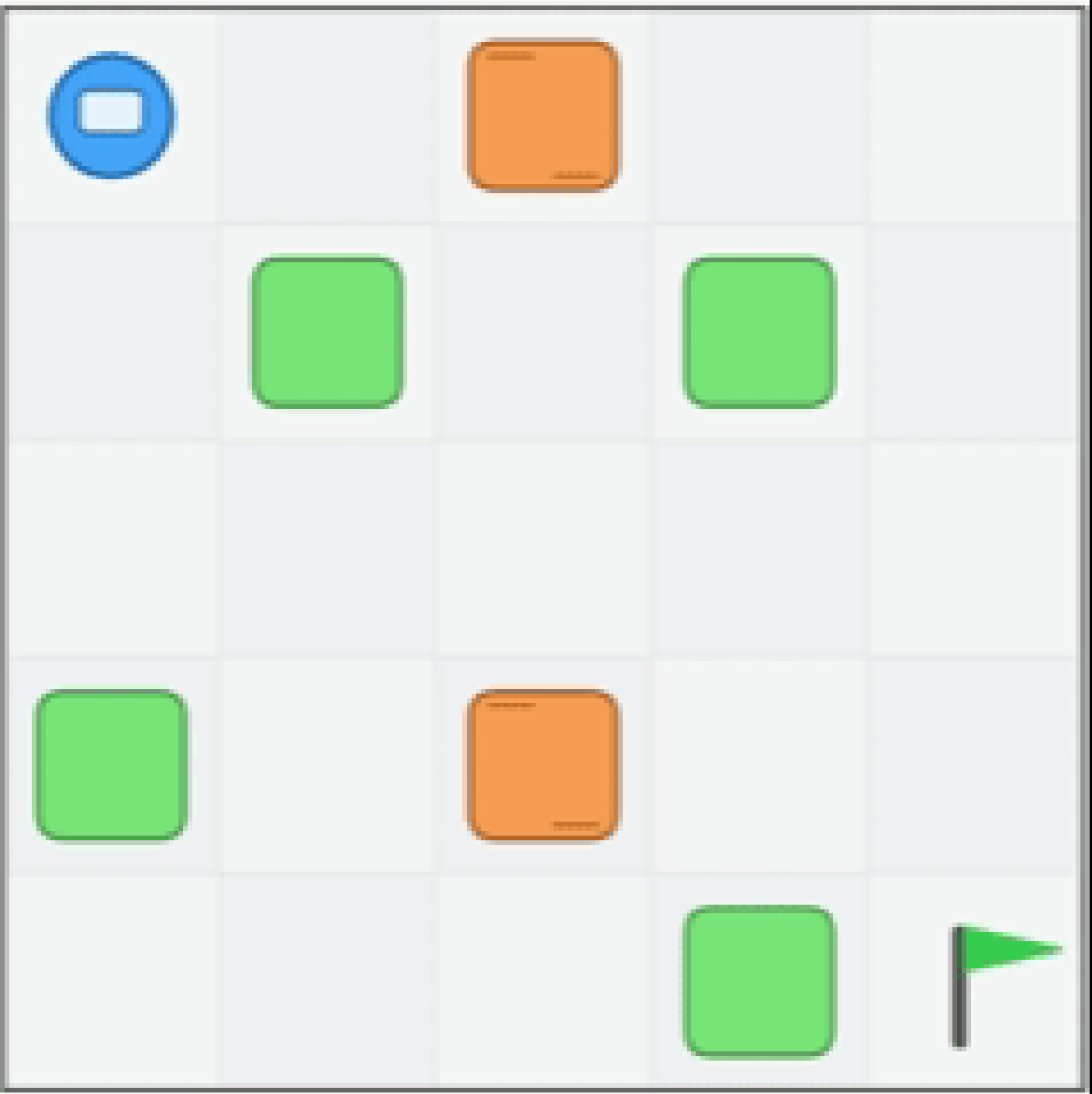
↗ Reencaminhamento

Adaptação imediata no passo 4

Conclusão: O agente adapta-se imediatamente ao distúrbio e encontra um caminho alternativo, demonstrando comportamento reativo e planejamento dinâmico.



Validação: Testes A, B e C



Teste C — Obstáculos Dinâmicos

ROBUSTEZ

Robustez estatística com múltiplas execuções independentes

Métricas de Desempenho

Taxa de Sucesso	100%
ep_len Médio	12.1
ep_len Mediano	11

Análise de Colisões

Colisões com Parede	11
Revisitas	18

Total de eventos: 29 ocorrências

Interpretação: O agente mantém desempenho robusto, atingindo o objetivo em todos os cenários testados.
A mediana igual à rota ótima (8 passos) demonstra que, na maior parte dos casos, o agente encontra trajetórias bastante eficientes.



Capacidades Demonstradas



Convergência Rápida

Políticas estáveis alcançadas após poucos milhares de episódios, com comprimentos convergindo para 8-12 passos após apenas 200 episódios iniciais.



Taxa de Sucesso: 100%

Desempenho perfeito sob dinâmica aleatória de obstáculos, atingindo o objetivo em todos os cenários testados.



Trajetórias Curtas

Execução de caminhos frequentemente iguais ou próximos do ótimo em ambientes estáticos, demonstrando alta eficiência.



Adaptação a Bloqueios

Reencaminhamento eficiente diante de obstáculos inesperados, com baixo custo adicional e resposta imediata.



Robustez Estatística

Desempenho consistente mantido em testes prolongados com múltiplas variações do ambiente, validando a generalização do modelo.

Conclusões e Principais Aprendizados

Síntese do projeto

Este estudo validou a eficácia do modelo A2C-LSTM para tarefas de navegação em ambientes dinâmicos e parcialmente observáveis. A combinação de memória temporal, recompensas densas e estabilização do treinamento provou ser uma estratégia poderosa.

Principais Aprendizados

1. **Memória é Essencial para Incerteza**

Em cenários POMDP, a memória (via LSTM) não é um luxo, mas um requisito fundamental para o agente construir um entendimento interno do ambiente e agir de forma inteligente.

2. **Recompensas Moldam o Destino**

A engenharia de recompensas impacta diretamente a velocidade de aprendizado e o comportamento final. Funções de recompensa que penalizam o tempo ("Rapidez") são altamente eficazes para gerar políticas eficientes em ambientes dinâmicos.

3. **"Convergência Rápida é Possível"**

Mesmo em um problema complexo, a arquitetura e a estratégia de recompensa corretas podem levar a uma convergência estável em poucas centenas de episódios.

4. **"A Adaptação é a Verdadeira Medida de Inteligência"**

O agente demonstrou capacidade de se adaptar a bloqueios inesperados, validando que a política aprendida é generalizável e não apenas uma solução superajustada.

Os resultados fornecem uma base sólida para o desenvolvimento de agentes autônomos mais complexos, capazes de operar de forma robusta e eficiente no mundo real.

Perspectivas Futuras e Referências

Aplicações

- Robótica industrial
- Drones autônomos
- Veículos autônomos
- NPCs em jogos

Extensões

- Entornos 3D contínuos
- Sensores reais (LIDAR)
- Sistemas multiagente
- Foco em segurança crítica

Referências

Sutton & Barto (2018), Grondman et al. (2012)

Código Disponível

GitHub:

github.com/luisenriqueCR/TP557

Perguntas?

